BITS PILANI WILP | AIMLCZG523

MLOps Assignment 01

Heart Disease Classification

End-to-end model development, CI and local Kubernetes deployment

Project objective

Build a reproducible binary classifier using the UCI Heart Disease dataset, track experiments, validate code with automated checks, package inference behind a REST API, and deploy locally on Kubernetes.

Student: Aditya Mishra

Repository: https://github.com/2025af05139-alt/heart-disease-mlops

Prepared: 20 October 2026

EDUCATIONAL DEMONSTRATION ONLY - NOT FOR MEDICAL USE

Report includes reproducible instructions, experimental results and captured implementation evidence.

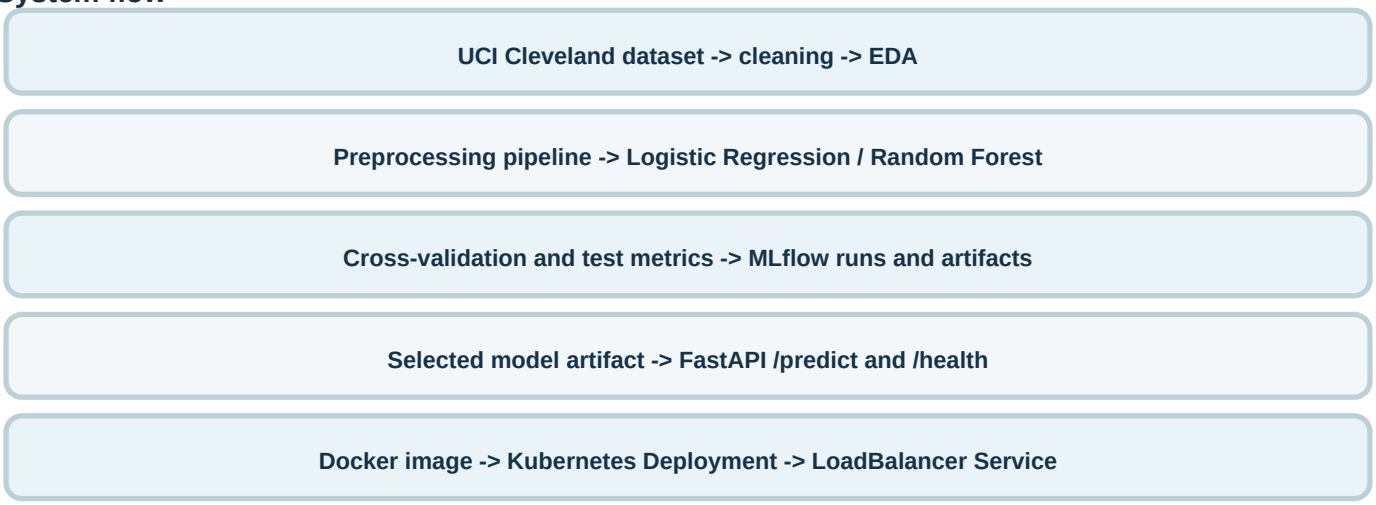
1. Executive summary and architecture

This project implements the main stages of a practical MLOps workflow: data acquisition and cleaning, exploratory data analysis (EDA), model comparison, experiment tracking, automated tests, API packaging, containerization and local Kubernetes deployment.

Implemented technologies

Area	Implementation
Language / environment	Python 3.11; requirements.txt
Data and visualization	pandas, NumPy, matplotlib, seaborn
Model development	scikit-learn pipelines; Logistic Regression and Random Forest
Tracking	MLflow with local SQLite backend
Serving	FastAPI + Uvicorn
Quality checks	pytest and Ruff
Container / orchestration	Docker; Minikube on Docker Desktop
CI	GitHub Actions workflow: .github/workflows/ci.yml

System flow



GitHub Actions independently runs linting, tests, data preparation, EDA, model training and saves workflow artifacts.

2. Dataset acquisition and exploratory analysis

Dataset source: UCI Machine Learning Repository, Heart Disease dataset (ID 45), using the Cleveland subset. The download and preparation logic is implemented in src/data.py, and EDA generation is implemented in src/eda.py.

Recorded data checks

Check	Recorded result
Rows / columns	303 rows / 14 columns
Duplicate rows	0
Missing values	ca: 4; thal: 2
Target counts	Class 0: 164; class 1: 139
Target mapping	Original 0 -> class 0; original 1-4 -> class 1

Exploratory visualizations

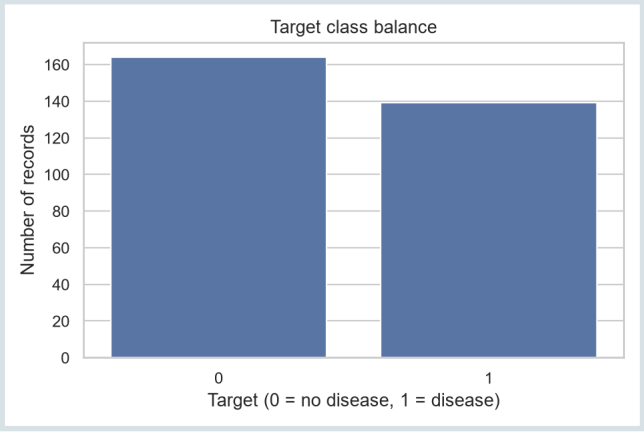
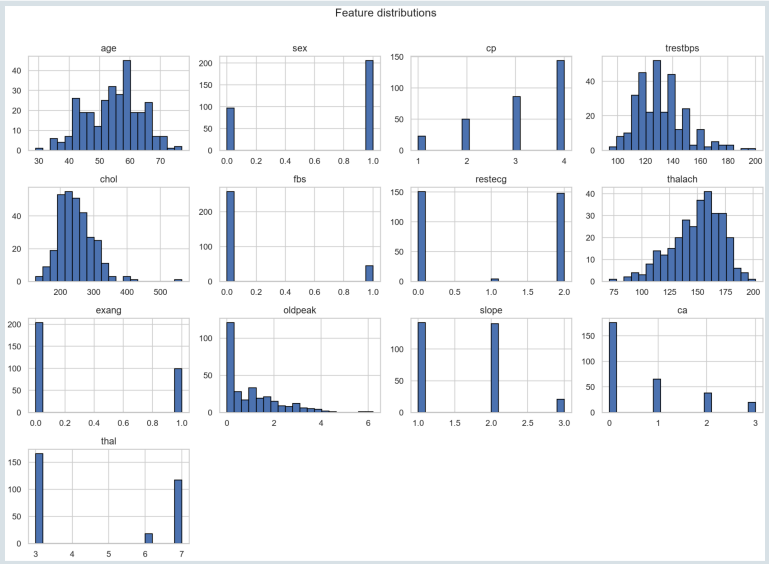


Figure 1. Class balance in the processed Cleveland subset.



odel variables.

Figure 3. Feature distributions generated by the EDA script.

3. Model development and evaluation

The training script evaluates two classification approaches. Numeric and categorical preprocessing are fitted within scikit-learn pipelines. A stratified 80/20 train/test split was used (242 training rows and 61 test rows), with five-fold StratifiedKFold cross-validation. The selection rule is mean cross-validation ROC-AUC.

Recorded metrics

Metric	Logistic Regression	Random Forest
Mean CV ROC-AUC	0.9025	0.8964
CV ROC-AUC std. dev.	0.0161	0.0351
Test accuracy	0.8852	0.8689
Test precision	0.8387	0.8125
Test recall	0.9286	0.9286
Test ROC-AUC	0.9665	0.9491

Selection decision

Logistic Regression was selected because its mean cross-validation ROC-AUC (0.9025) exceeded the Random Forest result (0.8964). The held-out test set was used for reporting final evaluation metrics rather than selecting the model.

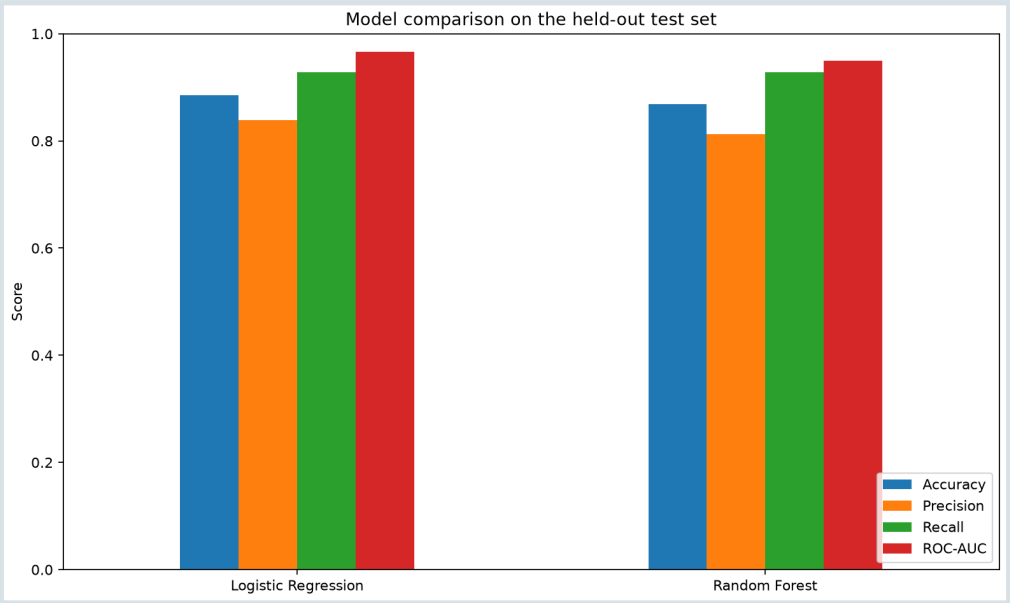


Figure 4. Model comparison plot generated by the training pipeline.

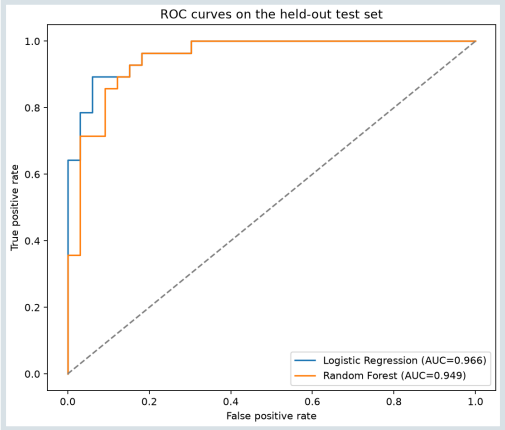


Figure 5. ROC curves on the held-out test set.

4. Experiment tracking and artifacts

MLflow experiment name: **Heart Disease Classification**. The training script initializes the tracking URI from the project root and logs separate runs for Logistic Regression and Random Forest, including parameters, cross-validation metrics, test metrics, comparison outputs, evaluation plots and model artifacts.

Tracked outputs include

- Model parameters and run metadata, including data/selection context.
- Cross-validation and held-out test metrics, including ROC-AUC, accuracy, precision and recall.
- Model comparison table and evaluation visualizations.
- Serialized model artifact for inference.

MLflow user interface evidence

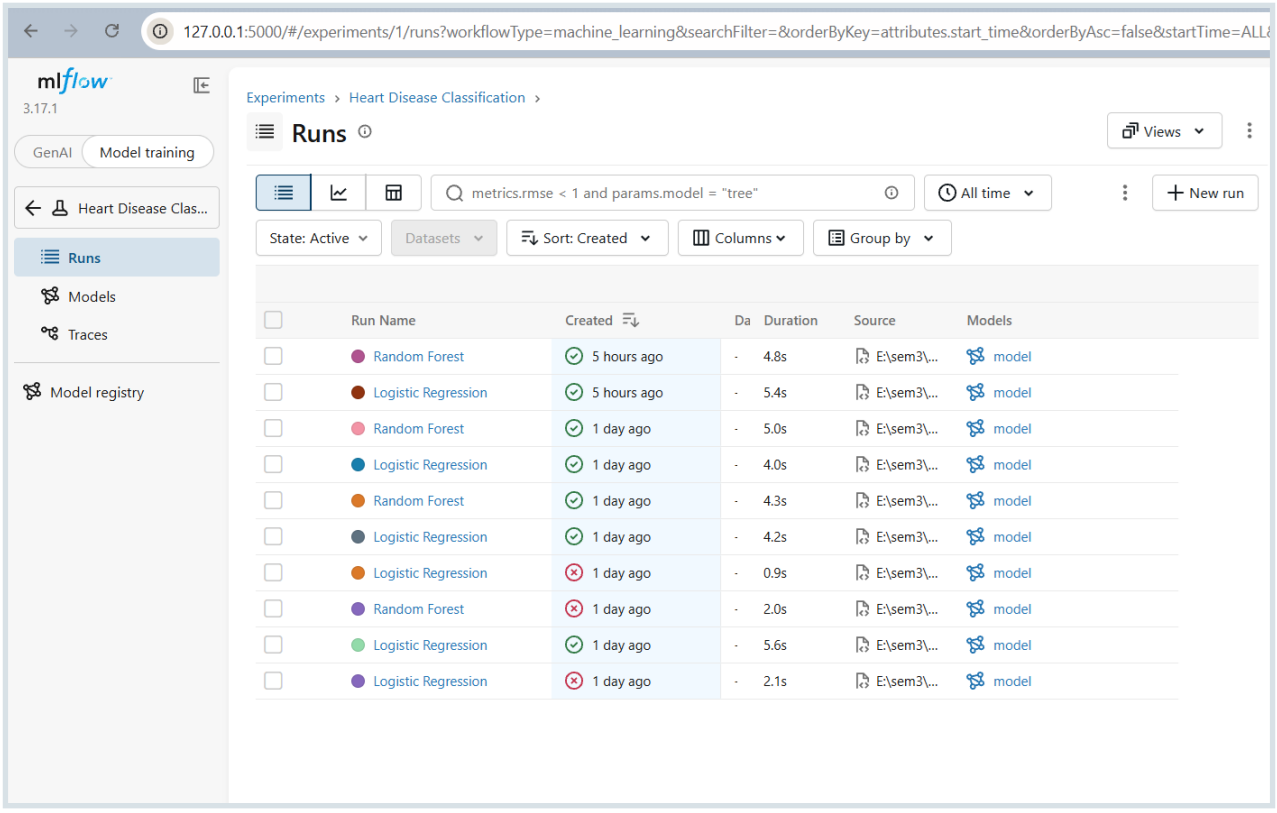


Figure 6. MLflow experiment runs for the two candidate models. The run table is shown from the local MLflow UI.

For reproduction on the development machine, start the MLflow server using the SQLite database path documented in README.md. The UI at http://127.0.0.1:5000 is local to the machine and is not a public URL.

5. Model packaging and API validation

The selected preprocessing-and-estimator pipeline is saved as artifacts/models/heart_disease_best_model.joblib. The FastAPI implementation is src/api.py and exposes a request/response interface for inference.

Endpoint	Purpose
GET /	Service information
GET /health	Health and model-loaded status
POST /predict	Return predicted class, label, probability and model name
GET /docs	Interactive OpenAPI / Swagger UI

Validation evidence

A sample JSON request through the Kubernetes LoadBalancer returned HTTP 200. The shown response returned prediction 0 (negative) and a probability estimate of approximately 0.195 for the positive class, with the educational-use disclaimer. This is a software test example and is not a clinical assessment.

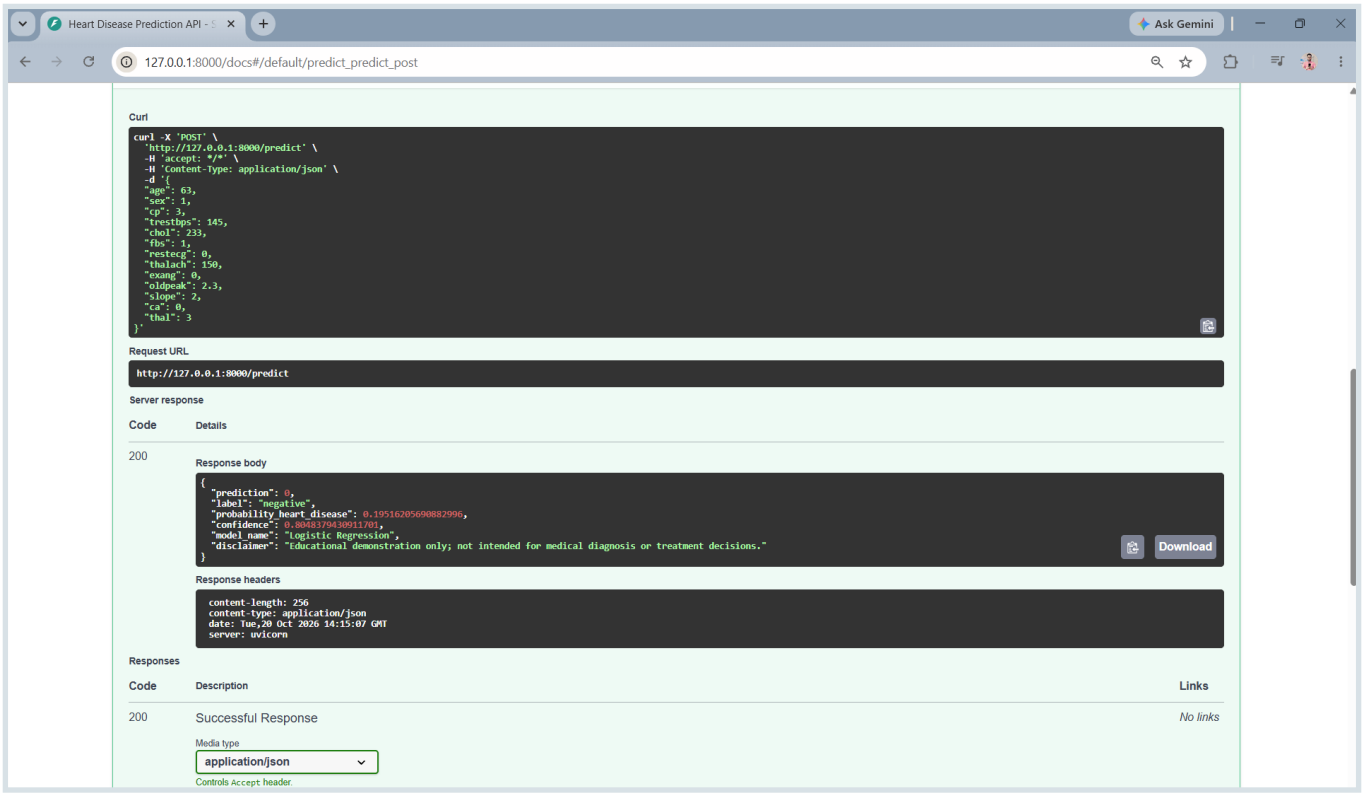


Figure 7. Swagger UI response for POST /predict with HTTP 200.

The model is intended only for educational demonstration; its output is not clinically validated and must not guide diagnosis or treatment.

6. Containerization and Kubernetes deployment

The API is packaged in the Docker image heart-disease-api:1.0. Dockerfile installs requirements, copies the API source, selected model and metrics metadata, and starts Uvicorn on container port 8000.

Local Docker commands

```
docker build -t heart-disease-api:1.0 .
docker run --rm -p 8000:8000 heart-disease-api:1.0
```

The Kubernetes manifest in k8s/deployment.yaml defines one replica, HTTP health probes and resource requests/limits. k8s/service.yaml exposes port 8000 using a LoadBalancer Service. Minikube is configured with Docker Desktop as driver; the minikube tunnel process must remain running for the local external IP to be reachable.

Recorded cluster state

- Deployment READY: 1/1; AVAILABLE: 1.
- Pod READY: 1/1; STATUS: Running; restarts: 0.
- Service type: LoadBalancer; external IP: 127.0.0.1; service port: 8000.

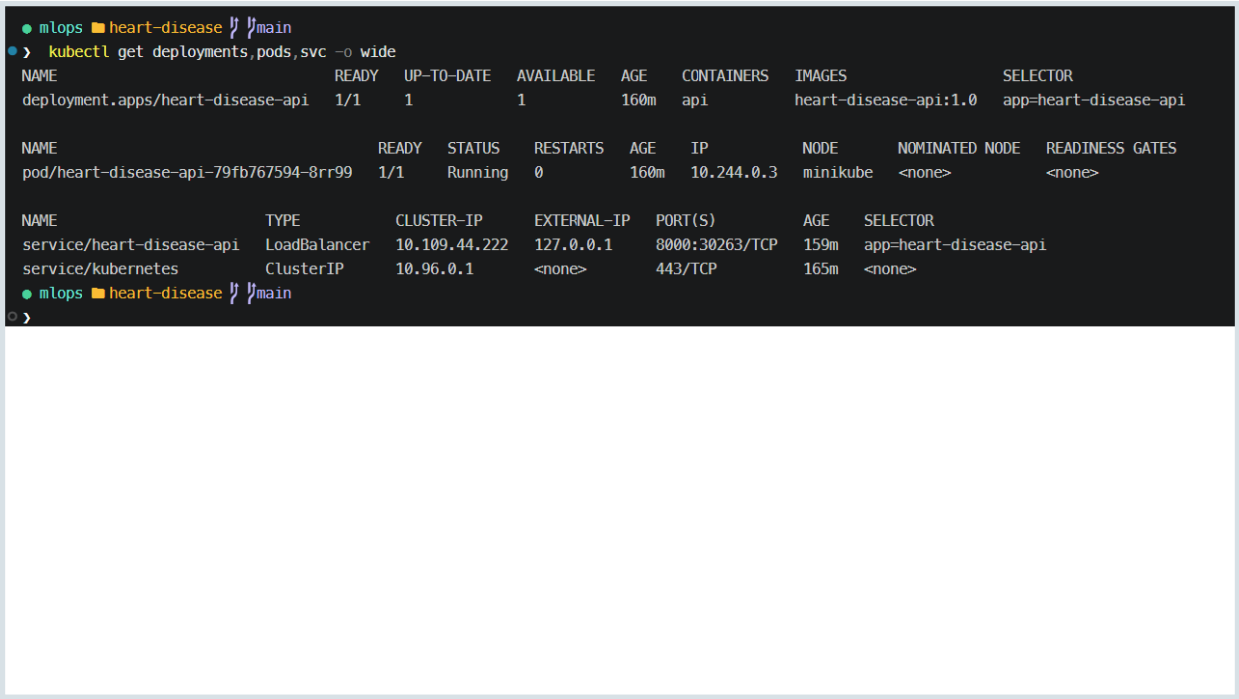


Figure 8. kubectl output showing Deployment, Pod and LoadBalancer Service.

7. Logging and operational checks

The API logs completed requests with method, request path, HTTP status code and elapsed duration. The example log output includes repeated successful GET /health checks generated by Kubernetes probes and a successful POST /predict request.

Operational evidence visible in the logs

- Repeated GET /health requests completed with status_code=200.
- A prediction request completed with method=POST, path=/predict and status_code=200.
- Request durations are recorded in milliseconds; the observed prediction request took about 33 ms in the captured example.
- Readiness and liveness probes use the /health endpoint.

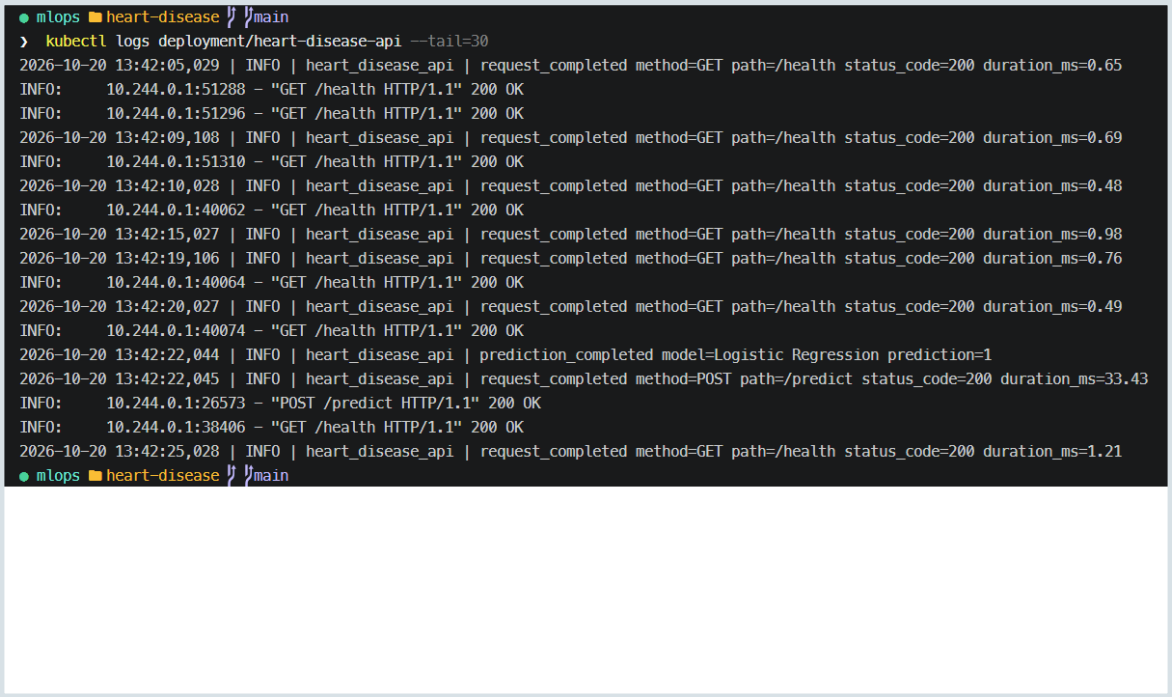


Figure 9. Kubernetes pod logs showing health-probe traffic and a successful prediction request.

Monitoring limitation: this implementation demonstrates application request logs and Kubernetes health probes. A dedicated Prometheus/Grafana dashboard is not part of the verified current setup. This limitation is stated transparently rather than claiming an unimplemented dashboard.

8. Automated testing and GitHub Actions

The GitHub Actions workflow is stored at .github/workflows/ci.yml. It is triggered by pushes and pull requests targeting main, and supports manual execution. The job runs on ubuntu-latest with Python 3.11.

Configured workflow steps

- Install project dependencies from requirements.txt.
- Run Ruff linting on src and tests.
- Run pytest and create a JUnit XML test report.
- Download/preprocess data and generate EDA outputs.
- Train and evaluate the candidate models.
- Upload test results, model artifacts, reports and MLflow tracking outputs as workflow artifacts.

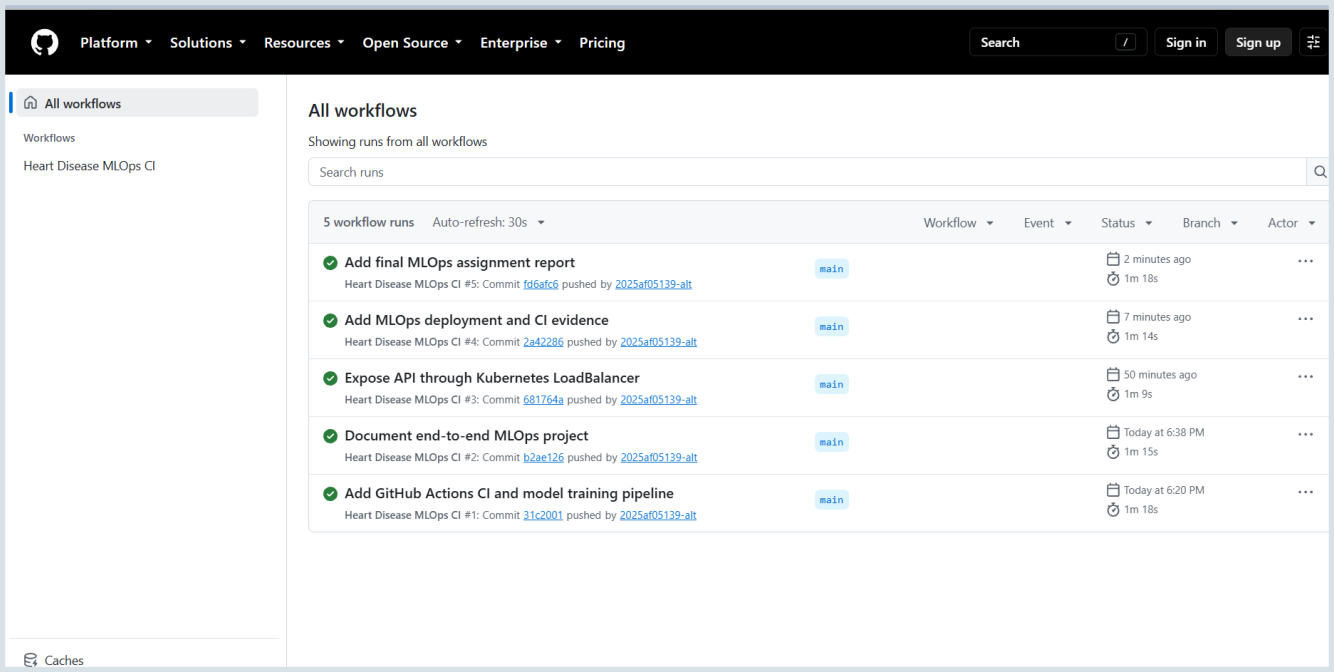


Figure 10. GitHub Actions page showing successful workflow runs for CI, documentation, deployment exposure and evidence/report commits.

Local quality checks recorded: 7 pytest tests passed and `ruff check src tests` reported all checks passed. Five successful Actions runs were visible in the captured GitHub page after the final report and evidence commits.

9. Reproduction, submission links and conclusion

Reproduction on a compatible local Windows environment

- **1.** Install Python 3.11 and Docker Desktop. Install Minikube and kubectl; ensure Docker Desktop is running.
- **2.** Install dependencies: `python -m pip install -r requirements.txt` in the project environment.
- **3.** Prepare the dataset: `python -m src.data`; create EDA outputs: `python -m src.eda`.
- **4.** Train and log: `python -m src.train`; run checks: `python -m pytest -q` and `ruff check src tests`.
- **5.** Build the API image: `docker build -t heart-disease-api:1.0 .`.
- **6.** Start Minikube, load the image, apply `k8s/deployment.yaml` and `k8s/service.yaml`, and verify with `kubectl get deployments,pods,svc -o wide`.
- **7.** Run `minikube tunnel` in a separate terminal and keep it open. Validate `/health`, `/docs` and `/predict` at `http://127.0.0.1:8000`.

Repository and CI links

Source repository: https://github.com/2025af05139-alt/heart-disease-mlops

Actions runs: https://github.com/2025af05139-alt/heart-disease-mlops/actions

Limitations and responsible use

The API is deployed locally, not on a public cloud. The 127.0.0.1 address is local to the development computer and is not accessible externally. The Minikube tunnel and Docker Desktop must be running for the demonstrated local endpoint to work. The model uses a small historical dataset, has not been clinically validated, and must not be used to make medical decisions. The current monitoring evidence is request logs and health probes; a dedicated Prometheus/Grafana dashboard was not verified.

Conclusion

The project demonstrates the core lifecycle of a small MLOps application: data preparation, EDA, model experimentation, tracked training, automated quality checks, API packaging and local Kubernetes serving. Source code, manifests, reports and supporting screenshots are maintained in the linked GitHub repository.